

Why I use API keys for book pricing, and how to get them

Author: Pankaj Yadav, Forward Deployed Software Engineer · github.com/Pankkaj64 · linkedin.com/in/pankkaj64

Why prices come from an API and not from the AI model

The brief says a price with no retrievable source is a fabricated price. So the language models in this project are never asked "how much is this book". Every price must come from a real listing with a link and a date that an adjuster can open. Two public sources do that:

- **eBay Browse API** gives real listings for the book, new and used, in the claim country. New listings give the replacement cost, used listings give the used value.
- **Google Books API** gives the publisher's list price for the country, plus the catalogue record (title, authors, publisher, ISBN) used to confirm the identification.

Both are free, but both need a key. Without a key they do not work for long:

Source	Without a key	With a free key
Google Books	requests share one public quota with everyone on the internet; it is usually already used up, so calls fail with HTTP 429 within minutes	your own quota, about 1,000 requests per day
eBay Browse	does not work at all; every call needs an application token	about 5,000 calls per day; the backend requests tokens itself and refreshes them every 2 hours

One sweep of 60 books uses roughly 60 to 120 calls per source, so the free limits are enough for several sweeps a day. The key is only used by the backend, from `backend/.env`. It is never committed and never shown in the UI or the packet.

How the key reaches the code

```
backend/.env          GOOGLE_BOOKS_API_KEY=...   EBAY_CLIENT_ID=...
EBAY_CLIENT_SECRET=...
|
backend/app/config.py  reads them with os.getenv (empty = source disabled)
|
backend/app/logic/providers.py
    GoogleBooksProvider -> sends the key in the x-goog-api-key header
    EbayAuth            -> exchanges id + secret for a 2-hour token, cached
|
GET /api/health        shows "google_books": true / "ebay": true when configured
python tools/check_price_sources.py  makes one real search per source to prove it
works
```

Google Books key, step by step (about 10 minutes)

1. Open <https://console.cloud.google.com> and sign in with any Google account.
2. At the top, click the project dropdown, then **New project**. Give it any name (for example `library-claim`) and click **Create**. Wait a few seconds and select it.
3. In the left menu open **APIs & Services** → **Library**.
4. Search for **Books API**, open it, click **Enable**.
5. In the left menu open **APIs & Services** → **Credentials**.
6. Click **Create credentials** → **API key**. A key starting with `AIza...` appears. Copy it.
7. Recommended: click the key name, under **API restrictions** choose **Restrict key**, tick **Books API** only, save. Then the key cannot be used for anything else if it leaks.
8. Put it in `backend/.env`:

```
GOOGLE_BOOKS_API_KEY=AIza...your key...
```

No billing account is needed. If the console shows a banner asking to set up billing, ignore it.

eBay keys, step by step (about 30 minutes)

1. Open <https://developer.ebay.com> and click **Register** (top right). You can use an existing eBay buyer account or create a new one. Confirm the email eBay sends.
2. After signing in, click your name (top right) → **Your Account**.
3. In the left menu click **Application Keys**.
4. You see two sections: **Sandbox** and **Production**. Use **Production**. Sandbox keys work but only return fake test listings with made-up prices.

5. Under Production click **Create a keyset**. If eBay shows a licence agreement, accept it. If it asks for an application name, type anything (for example `library-claim-agent`).
6. The keyset shows three values:
 - **App ID (Client ID)** → this is `EBAY_CLIENT_ID`
 - **Cert ID (Client Secret)** → this is `EBAY_CLIENT_SECRET`
 - Dev ID → not needed
7. Put them in `backend/.env`:

```
EBAY_CLIENT_ID=...App ID...
EBAY_CLIENT_SECRET=...Cert ID...
EBAY_ENV=production
```

Nothing else has to be enabled. The Browse API only needs the basic scope (`https://api.ebay.com/oauth/api_scope`), which every production keyset has. The backend gets a token with the client-credentials grant and refreshes it before it expires; you never have to paste a token by hand.

Check that it works

```
cd backend
# restart uvicorn after editing .env, then:
python tools/check_price_sources.py GB
python tools/check_price_sources.py AE
```

Good output looks like:

```
google_books: status=candidates quotes=6
ebay: status=ok quotes=2
  replacement 12.99 GBP condition=New match=0.95 https://www.ebay.co.uk/itm/...
  used        6.50 GBP condition=Good match=0.95 https://www.ebay.co.uk/itm/...
fx: USD->GBP 0.7565 (Thu, 08 Oct 2026)
OK: every source answered
```

`GET http://127.0.0.1:8000/api/health` must show `"google_books": true` and `"ebay": true`.

What to expect from each source

- Google Books mostly prices **ebooks** (Google Play). The valuer records them as candidates but rejects them as a physical replacement cost, with the reason written in `price_details`, unless `PRICING_ALLOW_EB00K_PROXY=true` is set in `.env`. So Google Books alone leaves most replacement costs blank; its main value is the catalogue record.

- eBay gives the physical prices: the median of the matching fixed-price listings, with every listing kept as a comparable. Books with no listing in the claim country are priced from the US marketplace and converted with the day's ECB rate, marked `converted: true`.
- A listing is used only when its title and author match the identified book with a score of at least 0.8 (`PRICING_MIN_MATCH`). A wrong edition is worse than a blank.

Keep the keys safe

- `backend/.env` is in `.gitignore`. Never put keys in `.env.example`, the README or a commit.
- For the submission, send the keys in the email, not in the repository, or let the tester create their own with the steps above.
- If a key is ever pasted somewhere public (chat, screenshot), regenerate it in the console.